

SLASH ROLLER: ARENA

Game Design Document — Final Draft (Capstone Cut)

Course: Multi-Agent AI for Game Development (ELVTR)

Assignment: #02 — GDD Final Draft

Author: Juan Diego Lugo

Date: 22 July 2026

Supersedes: Assignment #1 first draft (*Last Call*) — see Revision Note, §8

1. Executive Summary

Slash Roller: Arena is a multiplayer melee-combat deathmatch game with souls-weight fighting: committed light/heavy strikes, block, parry, dodge, and a stamina bar, plus one magic slot per loadout — *For Honor*'s dueling weight at deathmatch pace. Matches are fought in one small arena.

- **Win condition:** Most kills when the 8-minute match timer expires.

FFA ties break by fewest deaths, then sudden death (first kill wins). In Team Deathmatch, the higher team kill total wins.

- **Loss condition:** Any other scoreline at the timer.

Modes: Free-for-all **Deathmatch** (2–4 fighters) and **Team Deathmatch** (2v2) — PvP over a **Steam listen server** (host + invite), and **PvE:** bots fill any empty slot and power a full solo offering, so a match can always start.

Platform & format: Unreal Engine 5.8, pure native C++ on the Gameplay Ability System (GAS), server-authoritative with client prediction. Audio is

MetaSounds (engine-native — no external middleware). Environment art from marketplace assets; **all gameplay code written in-house.**

Team & timeline reality: One principal engineer / technical director plus a multi-agent AI crew, 5 weeks, on top of an already-shipped combat foundation (GAS core, soft-lock melee, input buffer, Steam sessions).

Steam demo build is the exit criterion — and the meta-goal is the method itself: prove that a senior engineer directing an agent crew can ship a small, professionally complete game in weeks.

Elevator pitch: *Dark Souls* combat weight at *Quake* deathmatch pace, in one arena, built by one engineer and an AI crew in five weeks.

2. Game Mechanics (Core Loop)

2.1 The Match Loop

1. **Host or join** — host a listen server and invite via Steam, or start

solo; bots fill remaining slots. Pick a loadout: weapon archetype + one magic slot.

2. **Fight** — hunt opponents through the arena; every takedown scores a kill on the board.

3. **Die and respawn** — on death, a 5-second respawn timer, then re-enter at the spawn point farthest from active combat. Loadout (including the magic slot) may be swapped while dead.

4. **Timer expires** — scoreboard, winner, one coach line on your match habits, rematch prompt ("run it back?").

2.2 Combat Economy — two currencies

Commitment: every attack locks the attacker into recoverable animation frames. Lights are fast, low damage, safe; heavies are slow, high damage, and punishable; block negates lights but is broken by heavies; parry (a timed block) staggers the attacker for a guaranteed punish; dodge i-frames through anything but costs the most stamina.

Stamina: attacks, dodges, and blocking drain a regenerating stamina bar. At zero stamina the fighter is **winded** — no attacks or dodges until 30% recovery. Stamina is the souls-feel governor: it forces pacing, punishes button-mashing, and makes aggression a spend decision.

The magic slot (projectile or AoE, by loadout) is the momentum swing on a 12-second cooldown — the ranged answer in a melee game, deliberately scarce.

2.3 Deathmatch with souls combat — the design risk, named

Committed animations plus free-for-all means getting hit from behind is a real threat. This is handled by design, not denial: small fighter counts (2–4), one compact arena readable at a glance, the existing soft-lock / target-assist system for fast target switching, spawn placement biased away from active fights, and a short winded state so no fighter is helpless for long. **The W1 playtest gate tests exactly this:** if third-party ganks feel cheap, the tuning levers are arena size, fighter count, and stamina regen — not new systems.

2.4 Why It's a Game and Not a Brawl

Kills require winning committed exchanges: mashing drains stamina into the winded state, turtling loses to heavies and to the clock, and the parry punish makes predictability lethal. The triangle (light / heavy–block / parry–dodge) plus stamina management is the whole skill ceiling — deep enough to practice, small enough to ship.

3. Player Experience (What the Player Sees)

- **HUD:** health and **stamina** bars bottom-center; magic-slot icon with

GAS-driven cooldown sweep bottom-right; match timer and a compact kill-leaderboard top-center; kill feed top-right. Nothing else.

- **Soft-lock feedback:** the current target is marked by a subtle

under-feet ring; strikes visibly track it; a flick of the camera or input direction switches targets instantly (score-based target assist).

- **Souls-feel feedback:** hit-stop on heavy connects, a distinct parry

clang and slow-tick stagger, an audible winded gasp at zero stamina — all MetaSounds cues driven by gameplay events.

- **Death & respawn:** a 5-second death cam on your killer, loadout swap

buttons active while dead, respawn flash at re-entry.

- **Match end:** scoreboard with K/D per fighter, winner banner, one

coach line referencing your actual match data ("You were parried 6 times — stop opening with the same heavy."), rematch prompt.

- **Front end (CommonUI):** main menu → host / join / solo vs bots;

lobby with mode (FFA / 2v2), fighter count, and loadout pick.

4. AI / Agent Architecture

Two layers: a **dev-time crew** that builds content through the Claude + Unreal Engine MCP pipeline, and **one runtime agent** kept strictly out of the simulation. Bots are deterministic C++ — the LLM never steers a fighter.

4.1 Dev-Time Crew (Claude + Unreal MCP, in-editor)

Agent	Role (one sentence)	Output format
Arena Architect	Blocks out the arena in-editor via UE MCP — geometry, spawn points, sightlines, cover — from a one-paragraph brief, then iterates from screenshots.	Editor edits + <code>arena_manifest.json</code> (bounds, spawn list, landmark names)
Bot Trainer	Generates and tunes bot combat parameters per difficulty tier against the stamina/commitment economy.	DT_BotTuning DataTable rows (aggression, parry %, reaction ms, stamina discipline)
Combat QA	Runs automated bot-vs-bot matches nightly, reads logs and screenshots, and files reproducible defect reports.	<code>qa_report.json</code> (defect, repro steps, severity)
Balance Analyst	Reads match telemetry and proposes tuning diffs when a loadout or option exceeds win-rate bounds (55% triggers review).	DataTable diff + one-paragraph rationale

Gameplay effect of the crew: the arena, the bots, and the balance the player experiences are agent-produced and human-approved. Every crew output is data (JSON/DataTables) reviewed before merge — agents never commit directly. This crew *is* the capstone's development thesis.

4.2 Runtime Agent (in the shipped game)

Caster Agent — the only model call in the running game, and it is never load-bearing.

- **Role sentence:** Generates color-commentary lines for notable kill-feed

events (streaks, parry kills, buzzer-beaters) during the match, and one telemetry-grounded coach line per player at match end.

- **Gameplay effect:** the arena feels broadcast — streaks get called,

humiliating parries get narrated — and every player leaves with one specific, data-backed habit to fix.

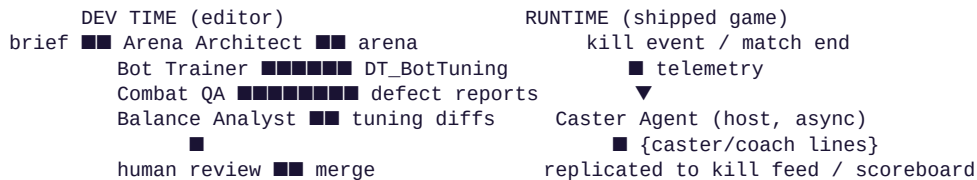
- **Output format:** {caster_line (≤ 18 words) | null} per event;

{coach_line (≤ 30 words, references ≥ 1 telemetry stat)} per player at match end.

- **Multiplayer correctness:** host-side async calls on kill events,

batched (≥ 10 s between calls, ≤ 12 calls/match); replies replicate as strings to the kill feed. The simulation never waits: a late or failed reply falls back to canned lines, and the kill feed's factual line ("A killed B") always renders instantly and locally.

4.3 Interaction Diagram



5. Technical Strategy

5.1 Stack (existing foundation in bold — already shipped and stable)

- UE 5.8, pure native C++, no-Tick event-driven architecture
- **GAS combat core: PlayerState-owned ASC, input buffer, prediction keys**
- ****Soft-lock melee (SoftLockTarget replicated source of truth) +**

score-based target assist + motion warping on strikes**

- **Steam OSS sessions (OSSessionsSubsystem), Perforce**
- **Motion-matched animation (AL Framework, custom AnimGraph nodes)**
- New this project: match/respawn flow, stamina attribute + winded state,

bot state machines, CommonUI front end, **MetaSounds** audio (gameplay- cue-driven, replacing the previous Wwise plan), Caster Agent HTTP client

- Environment art: marketplace asset packs (art is bought; code is not)

5.2 Model Assignment and Token Budget

Dev-time crew runs on Claude Code (Sonnet-class) under the existing 20-skill UE5 library and .mdc rules — development cost, not game cost.

Call	Model	Calls / match	Tokens in	Tokens out	Match total
Caster (kill events, batched)	Claude Haiku	≤ 12	600	60	$\approx 7,900$
Coach (match end, per player)	Claude Haiku	≤ 4	900	80	$\approx 3,900$

$\approx$ **\$0.01 per match** — effectively free. On timeout (> 3 s), error, or cap, canned lines ship; players without connectivity lose flavor, never function.

5.3 Constraints (named, with the design decision each caused)

1. **Constraint: multiplayer determinism.** An LLM cannot sit in the simulation path of a predicted, server-authoritative action game. *Therefore* the Caster Agent only decorates the kill feed asynchronously and coaches post-match; bots are deterministic C++; nothing the model says can change a fight in progress.
2. **Constraint: five weeks, one principal + crew.** New engineering surface must stay near zero. *Therefore* the game reuses the shipped combat stack wholesale; the only new gameplay systems are match/respawn flow, one stamina attribute, and bot state machines — and content (arena, tuning) is delegated to the dev-time crew.
3. **Constraint: small-PvP population risk.** An indie PvP game with an empty queue is dead at week two. *Therefore* bots fill every empty slot in every mode at launch (PvE is the same game, not a second game), and online is invite-first listen server rather than quickmatch-dependent.

5.4 Latency Plan

In-match combat uses the existing prediction/rollback stack — no model calls, no new latency surface. Caster calls are fire-and-forget with factual kill-feed lines rendering locally and instantly.

6. Scope & Development Plan

Shipped scope: FFA Deathmatch (2–4 fighters) and 2v2 Team Deathmatch; 1 arena; 2 loadout archetypes (blade, spellblade), 1 swappable magic slot each; stamina/winded system; bots at 3 tiers filling any slot; 8-minute matches with sudden-death tiebreak; CommonUI front end (menu, lobby, HUD, scoreboard); Caster Agent with canned fallback; MetaSounds combat audio; Steam demo build. **Nothing else.**

Explicitly cut: second arena, third archetype, ranked, quickmatch, Domination, dedicated servers/GameLift, cosmetics, achievements, 3v3+.

Week by week (each week ends runnable):

1. **W1 — Fun proof:** Arena Architect blocks out the arena via UE MCP;

match timer + kill scoring + respawn flow; stamina attribute + winded; FFA vs dumb bots, local. *Gate: winning a committed exchange feels good, and third-party ganks don't feel cheap.*

2. **W2 — Online:** Steam listen-server host/invite through `OSSessionsSubsystem`; second archetype; bot tiers 1–2; TDM team logic.

3. **W3 — Full loop:** telemetry + Caster/coach agent (canned lines first, API second); bot tier 3; loadout-swap-while-dead; MetaSounds combat cues.

4. **W4 — Balance & front end:** Balance Analyst pass on nightly Combat QA telemetry; CommonUI menu/lobby/scoreboard/kill feed; spawn-placement tuning.

5. **W5 — Ship:** Steam build + demo depot, soak tests, capstone presentation. Buffer for W1–4 slippage.

First cut if behind: Team Deathmatch (FFA alone carries the demo; team logic returns as the studio's fast-follow).

7. Success Criteria

- A stranger installs the Steam demo, fights bots inside 60 seconds, and hosts a friend without instructions.
 - Playtesters run it back unprompted at match end (the retention gate).
 - Every arena decision, bot tier, and balance change in the build traces to a named agent output that was human-reviewed — the crew-development thesis is demonstrable, not anecdotal.
-

8. Revision Note (Assignment #1 → Final Draft)

What changed: the capstone game. Assignment #1 specified *Last Call*, a browser-based AI interrogation game. The final draft replaces it with

Slash Roller: Arena, the scoped-down cut of the UE5 melee game my studio team is actively building toward Steam.

Why — the Class 03 method applied honestly: stress-testing the first draft surfaced that its strongest quality (a self-contained LLM game) was also its flaw as *my* capstone: it exercised none of my existing engineering and would be abandoned after the course. Meanwhile the studio project carried exactly the scope disease Class 03 warns about (a 122-item Early Access list). Applying the review discipline to the real project produced this cut: deathmatch-only modes on the shipped combat core, bots as a launch feature, and the multi-agent pipeline as both development method and runtime feature.

Meaningful changes visible in this draft (rubric: Revision & Growth):

1. **Scope creep addressed by deletion:** the studio vision's mode list,

class roster, and dedicated-server plan are cut to a shipped-scope paragraph ending in "Nothing else," with the first-cut feature named.

2. **Agent roles re-specified by output format:** four dev-time agents and one runtime agent, each with a named data deliverable (DataTables, JSON manifests) instead of free-text behavior.

3. **The LLM moved out of the hot path:** the first draft ran three model calls per player action; this draft runs zero calls in the simulation — async flavor and post-match coaching only, with canned fallbacks.